Přednáška 3: Simulování náhodných veličin

KMA/MSM Blanka Šedivá

verze 2026/2027
6. října 2026

Obsah

1. Úvod a generování pseudonáhodných čísel
2. Obecné metody simulování
3. Generování náhodných vektorů

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Simulování náhodných veličin

Proč potřebujeme simulovat náhodné veličiny?

- Chování řady statistických postupů (odhadů, testových statistik, procedur apod.) často nelze studovat exaktně (analyticky).
- Rozdělení může být příliš složité nebo prakticky nepoužitelné.
- Chování při konečných rozsazích výběru sledujeme pomocí simulací.
- Je-li posloupnost generována algoritmem, **není náhodná**. Důležité je, aby se **chovala jako náhodná**.

Simulování a zákon velkých čísel

- Nechť $X_1, X_2, \ldots, X_n$ je náhodný výběr z rozdělení náhodné veličiny X a $\mathbb{E}|g(X)| < \infty$. Pak podle silného zákona velkých čísel

$$\mu_n = \frac{1}{n}\sum_{i=1}^{n} g(X_i) \overset{n\to\infty}{\longrightarrow} \mu = \mathbb{E}g(X) \quad \text{(skoro jistě)},$$

  speciálně
  - $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} X_i \overset{n\to\infty}{\longrightarrow} \mathbb{E}X$,
  - $\frac{1}{n}\sum_{i=1}^{n}(X_i - \overline{X})^2 \overset{n\to\infty}{\longrightarrow} \operatorname{var} X$,
  - $\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}(X_i \le c) \overset{n\to\infty}{\longrightarrow} \mathcal{P}(X \le c)$ (relativní četnost),
  - median$(X_1, X_2, \ldots, X_n) \overset{n\to\infty}{\longrightarrow}$ median(X) (je-li medián jednoznačný).

Zákon velkých čísel v Pythonu

Monte Carlo odhady charakteristik $X \sim Exp(\lambda = \frac{1}{2})$, tj. $\mathbb{E}X = 2$, $\operatorname{var} X = 4$:

https://numpy.org/doc/stable/reference/random/generator.html

```
import numpy as np

rng = np.random.default_rng(42)   # generátor PCG64
n = 100_000
x = rng.exponential(scale=2.0, size=n)

print(x.mean())           # odhad E X        ~ 2
print(x.var())            # odhad var X      ~ 4
print(np.mean(x <= 1))    # odhad P(X <= 1)  ~ 0.39
print(np.median(x))       # odhad mediánu    ~ 2 ln 2
```

Přesnost odhadu průměru je řádu $\sigma/\sqrt{n}$ – pro desetinásobné zpřesnění potřebujeme stokrát více simulací.

FAKULTA APLIKOVANÝCH VĚD ZÁPADOČESKÉ UNIVERZITY V PLZNI

Generování náhodných (pseudonáhodných) čísel z rovnoměrného rozdělení

- lineární kongruentní generátor $x_i = (ax_{i-1} + c) \bmod m$, $u_i = x_i/m$ $(i = 1, 2, \dots)$
  jednoduchý, ale s periodou nejvýše m; body $(u_i, u_{i+1}, \dots)$ leží na nadrovinách
- multiplikativní rekurzivní generátor
  $x_i = (a_1 x_{i-1} + \cdots + a_k x_{i-k}) \bmod m \ \ (i = k, k+1, \dots)$
- kombinované generátory, např. MRG32k3a (L'Ecuyer):
  $x_i = (1403580\, x_{i-2} - 810728\, x_{i-3}) \bmod m_1$, $m_1 = 2^{32} - 209$,
  $y_i = (527612\, y_{i-1} - 1370589\, y_{i-3}) \bmod m_2$, $m_2 = 2^{32} - 22853$,
  $u_i = \frac{x_i - y_i + m_1}{m_1 + 1}$ pokud $x_i \le y_i$ a $u_i = \frac{x_i - y_i}{m_1 + 1}$ pokud $x_i > y_i$

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Současné generátory pseudonáhodných čísel

- současné generátory: Mersenne Twister (MT19937, perioda $2^{19937} - 1$), PCG64 (výchozí v NumPy), Philox (paralelní výpočty)
- kvalitu pseudonáhodných generátorů testuje řada testů (baterie TestU01, Diehard, NIST)
- seed = počáteční stav generátoru; stejný seed dává stejnou posloupnost, simulace je tak reprodukovatelná
- Více např. Dirk P. Kroese, Thomas Taimre, Zdravko I. Botev: Handbook of Monte Carlo Methods, Wiley, 2011. ISBN: 9780470177938

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Lineární kongruentní generátor v Pythonu

Pouze pro ilustraci – v praxi používáme knihovní generátory.

```
import numpy as np
from scipy import stats

def lcg(n, seed=1, a=1103515245, c=12345, m=2**31):
    x = np.empty(n, dtype=np.int64)
    x_prev = seed
    for i in range(n):
        x_prev = (a * x_prev + c) % m
        x[i] = x_prev
    return x / m                    # u_i v intervalu <0,1)

u = lcg(10_000)
print(u.mean(), u.var())            # ~ 1/2, ~ 1/12
print(stats.kstest(u, "uniform").pvalue)
```

FAKULTA APLIKOVANÝCH VĚD ZÁPADOČESKÉ UNIVERZITY V PLZNI

Obecné metody simulování náhodných veličin

- metoda inverzní transformace
  využívá znalost distribuční funkce
- zamítací metoda
  využívá znalost funkce hustoty
- doplňková přijímací metoda (acceptance-complement)
  $f = f_1 + f_2$, $f_1 \leq g$; kandidáta x z g přijmeme s pravděp. $f_1(x)/g(x)$, jinak generujeme z hustoty úměrné f_2
- kompoziční metoda
  lze využít, pokud umíme napsat funkci hustoty jako směs $f(x) = \sum_j p_j f_j(x)$ jiných funkcí hustoty
- transformační metoda
  hledaná veličina je funkcí veličin, které umíme generovat (např. Box-Mullerova transformace)

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Metoda inverzní transformace

Uvažujeme náhodnou veličinu X s distribuční funkcí $F(x)$ a zobecněnou inverzní distribuční funkcí $F^{-1}(t) = \inf\{x : F(x) \geq t\}$. Pokud U má rovnoměrné rozdělení na intervalu $(0, 1)$, pak náhodná veličina $X = F^{-1}(U)$ má distribuční funkci F.

Pro generování výběru $x_1, x_2, x_3, \ldots$ lze použít následující postup

- nageneruji u_i z rovnoměrného rozdělení na intervalu $(0, 1)$,
- $x_i = F^{-1}(u_i)$.

Metoda je vhodná pro situace, kdy umím explicitně vyjádřit funkci F^{-1} (exponenciální, Weibullovo, Cauchyho rozdělení, ...).
Funkci F^{-1} lze v některých případech nahradit vhodnou aproximací. Hodnotu x_i lze získat i numerickým řešením rovnice $F(x_i) = u_i$.
Pro diskrétní veličinu s hodnotami $a_1 < a_2 < \ldots$ volíme $x_i = a_j$, kde j je nejmenší index, pro který $u_i \leq p_1 + \cdots + p_j$.

FAKULTA APLIKOVANÝCH VĚD ZÁPADOČESKÉ UNIVERZITY V PLZNI

Metoda inverzní transformace v Pythonu

Exponenciální rozdělení: $F(x) = 1 - e^{-\lambda x}$, $F^{-1}(u) = -\frac{1}{\lambda} \ln(1-u)$.

```
import numpy as np

rng = np.random.default_rng(42)
n, lam = 100_000, 0.5
u = rng.random(n)
x = -np.log(1 - u) / lam                    # F^{-1}(u)
print(x.mean())                             # ~ 1/lambda = 2

# diskrétní veličina
hodnoty = np.array([0, 1, 2, 3])
p = np.array([0.1, 0.4, 0.3, 0.2])
idx = np.searchsorted(np.cumsum(p), rng.random(n),
                      side="right")
x_disc = hodnoty[idx]
print(np.bincount(x_disc) / n)              # ~ p
```

Zamítací metoda

Nechť X je náhodná veličina s hustotou $f(x)$ a nechť existuje dominující funkce hustoty $g(x)$, ze které umíme generovat, a konstanta $c \geq 1$ tak, že $f(x) \leq c \cdot g(x)$ pro všechna x. Pak lze pro generování výběru $x_1, x_2, x_3, \ldots$ použít následující postup

- nageneruji x_i s hustotou $g(x)$,
- nageneruji v_i z rovnoměrného rozdělení na intervalu $(0, 1)$ nezávisle na x_i,
- pokud platí $v_i \leq \frac{f(x_i)}{c \cdot g(x_i)}$, je x_i akceptováno do výběru, jinak je zamítnuto.

Pravděpodobnost přijetí je $\frac{1}{c}$, střední počet pokusů na jednu přijatou hodnotu je c. Konstantu proto volíme co nejmenší, $c = \sup_x \frac{f(x)}{g(x)}$. Funkci f stačí znát až na normovací konstantu.

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Zamítací metoda v Pythonu

$N(0,1)$ z Laplaceova rozdělení $g(x) = \frac{1}{2}e^{-|x|}$, $c = \sqrt{2e/\pi} \doteq 1{,}32$:

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
c = np.sqrt(2 * np.e / np.pi)
m = 50_000
x = rng.laplace(size=m)                    # kandidáti z g
v = rng.random(m)
prijato = v <= stats.norm.pdf(x) / (
                    c * stats.laplace.pdf(x))
z = x[prijato]

print(prijato.mean())                      # ~ 1/c = 0.76
print(stats.kstest(z, "norm").pvalue)      # test normality
```

Zamítací metoda pro funkci hustoty s omezeným nosičem

Pokud funkce $f(x)$ je nenulová na konečném intervalu (a, b) (označme $M = \max_{a \leq x \leq b} f(x)$), pak lze použít následující postup

- nageneruji u_i a v_i, dvě nezávislé veličiny z $R(0, 1)$,
- $x_i = a + (b - a) \cdot u_i$,
- pokud platí $v_i \cdot M \leq f(x_i)$, je x_i akceptováno do výběru.

Jde o speciální případ zamítací metody s $g \sim R(a, b)$ a $c = M(b - a)$.

Výhoda: není třeba znát distribuční funkci ani inverzní funkci F^{-1}.
Nevýhoda: pravděpodobnost přijetí je $\frac{1}{M(b-a)}$; pokud funkce $f(x)$ vymezuje v obdélníku $(a, b) \times (0, M)$ malou plochu, bude zamítnuto mnoho dvojic (u_i, v_i).
Modifikací je metoda sevření (squeeze): f obklopíme jednoduchými funkcemi $h_1 \leq f \leq h_2$ a o přijetí většinou rozhodneme bez výpočtu f.

Zamítací metoda s omezeným nosičem v Pythonu

Rozdělení *Beta*(2, 2): $f(x) = 6x(1-x)$ na $(0,1)$, $M = f(\frac{1}{2}) = 1{,}5$.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
a, b, M = 0.0, 1.0, 1.5
m = 50_000
u, v = rng.random(m), rng.random(m)
x = a + (b - a) * u
x_beta = x[v * M <= 6 * x * (1 - x)]

print(len(x_beta) / m)        # ~ 1/(M(b-a)) = 2/3
print(stats.kstest(x_beta, "beta",
                   args=(2, 2)).pvalue)
```

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Generování náhodných vektorů pomocí inverzní metody

Uvažujme dvourozměrnou náhodnou veličinu s distribuční funkcí $F(x, y)$. Označme $F_X(x)$ marginální distribuční funkci a $F(y \mid x)$ podmíněnou distribuční funkci.
Pro generování výběru (x_i, y_i) použijeme následující postup

- nageneruji u_i a v_i, dvě nezávislé veličiny z $R(0, 1)$,
- $x_i = F_X^{-1}(u_i)$,
- $y_i = F^{-1}(v_i \mid x_i)$ (inverze podmíněné distribuční funkce při pevném x_i).

Pro k-rozměrný vektor postupně generujeme x_1 z F_1, x_2 z $F(x_2 \mid x_1)$, $\ldots$, x_k z $F(x_k \mid x_1, \ldots, x_{k-1})$.

Příklad (P01b): $f(x, y) = \frac{1}{6}\left(\frac{x}{2} + \frac{y}{3}\right)$, $x \in (0, 2)$, $y \in (0, 3)$,

- $F_X(x) = \frac{x^2+2x}{8} \Rightarrow x = -1 + \sqrt{1 + 8u}$,
- $F(y \mid x) = \frac{xy/3+y^2/9}{x+1} \Rightarrow y = \frac{-3x+\sqrt{9x^2+36v(x+1)}}{2}$.

FAKULTA APLIKOVANÝCH VĚD ZÁPADOČESKÉ UNIVERZITY V PLZNI

Náhodný vektor inverzní metodou v Pythonu

Ověříme výsledky z P01b: $\mathbb{E}X = \frac{7}{6}$, $\mathbb{E}Y = \frac{7}{4}$, $\operatorname{cov}(X, Y) = -\frac{1}{24}$.

```
import numpy as np

rng = np.random.default_rng(42)
n = 100_000
u, v = rng.random(n), rng.random(n)

x = -1 + np.sqrt(1 + 8 * u)                    # F_X^{-1}(u)
y = (-3 * x + np.sqrt(9 * x**2
                + 36 * v * (x + 1))) / 2 # F^{-1}(v|x)

print(x.mean(), y.mean())      # ~ 7/6, 7/4
print(np.cov(x, y)[0, 1])      # ~ -1/24
```

Generování náhodných vektorů pomocí zamítací metody

Uvažujme dvourozměrnou náhodnou veličinu s funkcí hustoty $f(x, y)$.
Předpokládejme, že funkce je nenulová na omezené oblasti A.
Označme maximum $M = \max_{(x,y)\in A} f(x, y)$.
Pro generování výběru (x_i, y_i) použijeme následující postup

- nageneruji trojici x_i, y_i, w_i rovnoměrně z oblasti vymezené plochou A a výškou M, tj. (x_i, y_i) rovnoměrně z A a w_i z $R(0, M)$,
- pokud platí $w_i \leq f(x_i, y_i)$, je (x_i, y_i) akceptováno do výběru.

Pravděpodobnost přijetí je $\frac{1}{M \cdot |A|}$, kde $|A|$ je obsah oblasti A; s rostoucí dimenzí obvykle rychle klesá.
Rovnoměrné rozdělení na složité oblasti A získáme opět zamítáním z obdélníku, který A obsahuje.

Náhodný vektor zamítací metodou v Pythonu

Stejná hustota jako u inverzní metody, $A = (0,2) \times (0,3)$, $M = f(2,3) = \frac{1}{3}$.

```
import numpy as np

rng = np.random.default_rng(42)
m = 50_000
M = (2 / 2 + 3 / 3) / 6        # maximum f v bodě (2,3)
x = rng.uniform(0, 2, m)
y = rng.uniform(0, 3, m)
w = rng.uniform(0, M, m)
ok = w <= (x / 2 + y / 3) / 6
x, y = x[ok], y[ok]

print(ok.mean())                          # ~ 1/(M|A|) = 1/2
print(x.mean(), y.mean(), np.cov(x, y)[0, 1])
```

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Generování vícerozměrného normálního rozdělení

Využijeme vlastnost lineární transformace (P02b): je-li $\boldsymbol{Z} \sim N_k(\boldsymbol{0}, \boldsymbol{I})$, pak $\boldsymbol{X} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{Z} \sim N_k(\boldsymbol{\mu}, \boldsymbol{L}\boldsymbol{L}^T)$.

Pro generování výběru z $N_k(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ použijeme následující postup

- rozložíme $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^T$ (Choleského rozklad, $\boldsymbol{L}$ dolní trojúhelníková),
- nageneruji k nezávislých veličin $z_1, \ldots, z_k$ z $N(0, 1)$,
- položím $\boldsymbol{x} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$.

Místo $\boldsymbol{L}$ lze použít $\boldsymbol{\Sigma}^{1/2}$ nebo spektrální rozklad $\boldsymbol{\Sigma} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T$, $\boldsymbol{L} = \boldsymbol{V}\boldsymbol{\Lambda}^{1/2}$.

Box-Mullerova transformace: jsou-li $U_1, U_2 \sim R(0, 1)$ nezávislé, pak

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \qquad Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

jsou nezávislé $N(0, 1)$.

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Vícerozměrné normální rozdělení v Pythonu

https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.multivariate_normal.html

```
import numpy as np

rng = np.random.default_rng(42)
n = 100_000
mu = np.array([3.0, 2.0])
Sigma = np.array([[10.0, 5.0], [5.0, 5.0]])

L = np.linalg.cholesky(Sigma)          # Sigma = L L^T
Z = rng.standard_normal((n, 2))        # Z ~ N_2(0, I)
X = mu + Z @ L.T                       # X = mu + L Z
print(X.mean(axis=0), np.cov(X, rowvar=False))

X2 = rng.multivariate_normal(mu, Sigma, size=n)
```

FAKULTA APLIKOVANÝCH VĚD ZÁPADOČESKÉ UNIVERZITY V PLZNI